DUALE HOCHSCHULE SACHSEN

Verteilte Systeme

Wintersemester 2026

Dr. Michael Hofmann

DHSN: Glauchau

31.08.2026

Allgemeine Informationen

Ablauf

- 6 Termine mit je 2x2 LVS (2x90 Minuten) = 24 LVS

$\rightsquigarrow$ jeweils „Vorlesung“ + Beispiele und ggf. Übungsaufgaben

- Vorlesungsmaterialen $\rightsquigarrow$ OPAL: Theorie der Betriebssysteme und verteilten Systeme
- Abschlussklausur: 120 Minuten $\rightsquigarrow$ $\frac{1}{3}$ für Verteilte Systeme

Literatur

- Andrew S. Tanenbaum und Maarten van Steen. *Verteilte Systeme: Grundlagen und Paradigmen.* 2. Ausgabe, Addison Wesley, 2007.

Themenüberblick (aktueller Plan)

(1) 31.08.2026
- Einführung

(2) 10.09.2026
- Architekturen

(3) 14.09.2026
- Prozesse

(4) 14.10.2026
- Kommunikation

(5) 27.10.2026
- Synchronisation

(6) 02.11.2026
- Konsistenz und Replikation
- Fehlertoleranz
- Wiederholung, Fragen & Antworten

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Historische Entwicklung

Anfangszeit (1945-1985)

- Computer waren groß, teuer (teils > 10 Mio. \$) und selten
- Selbst Minicomputer kosteten zehntausende Dollar
- ⇝ Wenige Computer pro Organisation, keine Vernetzung, isoliertes arbeiteten

Technologische Revolution (Mitte der 1980er Jahre)

- Entwicklung leistungsfähiger Mikroprozessoren (8/16/32/64-Bit)
- Leistung eines Mainframes zu einem Bruchteil der Kosten
- Enorme Preis-/Leistungsgewinne (Faktor 10^{13} in 50 Jahren)

Netzwerk-Innovationen

- Local Area Network (LAN): verbindet hunderte Rechner in Gebäuden
- Wide Area Network (WAN): verbindet global Millionen von Rechnern

Entstehung verteilter Systeme

- Kombination vieler Rechner über Hochgeschwindigkeitsnetzwerke
- Abgrenzung zu zentralisierten Systemen (Einzelrechner mit Terminals)
- Netzwerke ermöglichen Koordination, Zusammenarbeit & neue Anwendungen

Definition: Verteiltes System

> *Ein verteiltes System ist eine Ansammlung unabhängiger Computer, die für ihre Nutzer wie ein einziges zusammenhängendes System erscheint.*

- Keine Einschränkungen zur Art der Computer oder ihrer Verbindung

⇝ Autonome Komponenten müssen Zusammenarbeiten

⇝ Herausforderungen für die Entwicklung von verteilten Systemen

Wichtige Eigenschaften

- Unterschiede zwischen Computern, ihre Kommunikation und interne Organisation sind vor dem Nutzer verborgen
- Einheitliche Nutzung unabhängig von Ort und Zeit
- Gute Erweiterbarkeit und Skalierbarkeit (im Prinzip)
- Kontinuierliche Verfügbarkeit trotz (temporärer) Störungen, Erweiterungen, oder Reparaturen

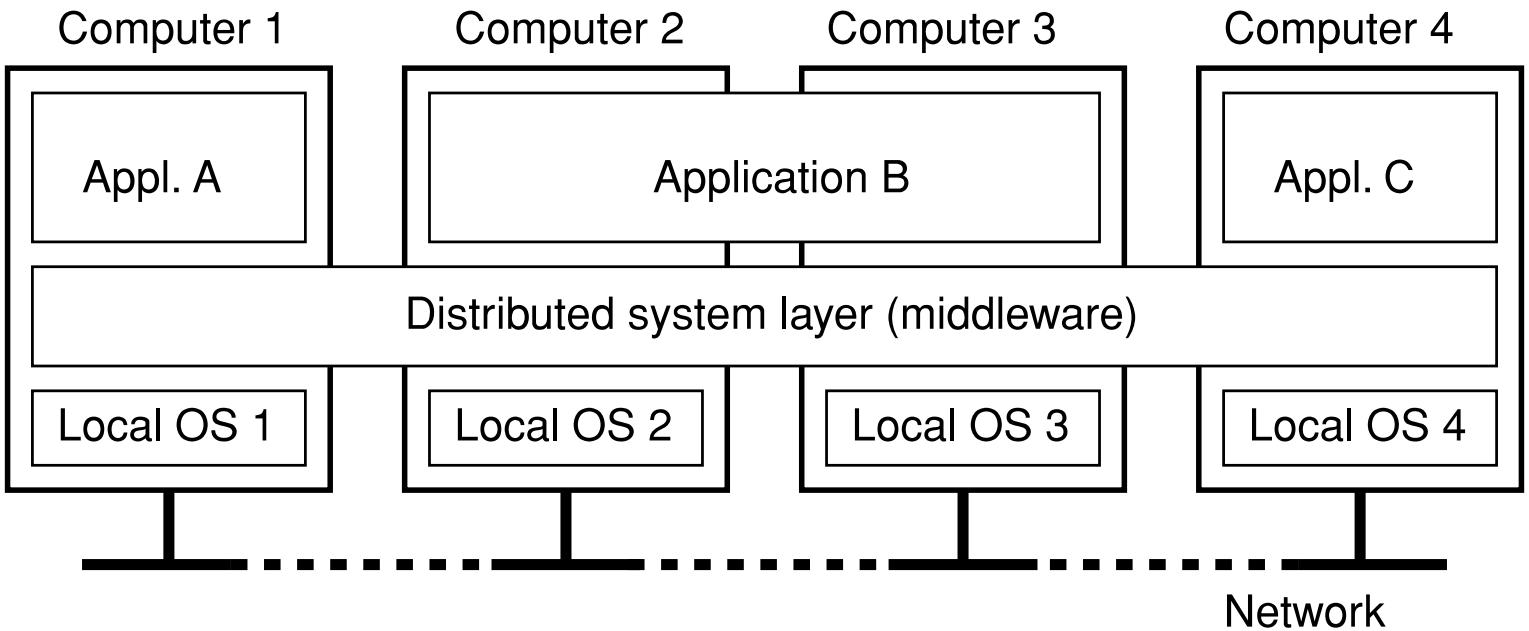
Allgemeiner Aufbau verteilter Systeme

Wie entwickelt man verteilte System, um die gegebenen Herausforderungen und Eigenschaften erfüllen?

Allgemeiner Aufbau verteilter Systeme

Wie entwickelt man verteilte System, um die gegebenen Herausforderungen und Eigenschaften erfüllen?

⇝ Nutzung einer Middleware Softwareschicht zur Behandlung der „Verteiltheit“



Inhalt

1. Einführung
    Ziele von verteilten System
    Arten von verteilten Systemen

Ziele von verteilten System

Warum sollte man den Aufwand betreiben ein verteiltes System zu entwickeln?

Ziele von verteilten System

Warum sollte man den Aufwand betreiben ein verteiltes System zu entwickeln?

⇝ Verteilte System ermöglichen es verschiedene Ziele zu erreichen

1. Verfügbarmachung von Ressourcen
   ⇝ Einfacher Zugriff und gemeinsame Nutzung von verteilten Ressourcen
2. Verteilte Transparenz
   ⇝ Physische Verteilung auf mehrere Computer wird verborgen
3. Offenheit
   ⇝ Standardisierte Beschreibung der Syntax (Wie?) und Semantik (Was?) von bereitgestellten Diensten
4. Skalierbarkeit ⇝ Erweiterbarkeit bzgl.
   - Anzahl von Nutzern und Ressourcen
   - geographischer und physischer Verteilung
   - Verwaltungsdomänen

Verfügbarmachung von Ressourcen

- Einfache, effiziente und gemeinsame Nutzung verteilter Ressourcen
- Typische Ressourcen: Computer, Speicher, Dateien, Webseiten, Netzwerke

Gründe

- Kosteneffizienz: Gemeinsame Nutzung teurer Hardware, z. B. Drucker, Hochleistungsrechner, Speichersysteme, spezialisierte Peripherie
- Kollaboration: Einfache Zusammenarbeit und Informationsaustausch, z. B. Datenaustausch per Internet, Zusammenarbeit per Groupware
- Zugänglichkeit: Reduzierung von physische Präsenz, z. B. Online-Handel

Herausforderungen

- Allgemeiner Schutz vor Abhören und Eindringen
- Schutz von sensiblen Daten, z. B. Passwörter, Zahlungsinformationen
- Ständig wachsende Größe von Daten und Informationen

⇝ Lösungsansätze: starke Verschlüsselung, sichere Authentifizierung, intelligente Filterung und Suche

Verteilte Transparenz

Definition: Transparenz (für Computersysteme)

1. Vorhanden, aber vom Nutzer nicht wahrgenommen
2. Durchsichtig, so dass Verhalten vom Nutzer nachvollzogen werden kann

Welche Definition von Transparent ist für verteilte Systeme wünschenswert?

Verteilte Transparenz

Definition: Transparenz (für Computersysteme)

1. Vorhanden, aber vom Nutzer nicht wahrgenommen
2. Durchsichtig, so dass Verhalten vom Nutzer nachvollzogen werden kann

Welche Definition von Transparent ist für verteilte Systeme wünschenswert?

Arten von Transparenz

- Zugriff $\rightsquigarrow$ Einheitlicher Zugriff unabhängig von technischer Repräsentation
- Verortung $\rightsquigarrow$ Physische Orte sind irrelevant, z. B. mittels logischer Benennungen
- Ortsveränderung (Migration & Relocation) $\rightsquigarrow$ Keine Beeinflussung nach bzw. während der Veränderung physischer Orte
- Replikation $\rightsquigarrow$ Identischer Zugriff auf *Originale* und *Kopien*
- Nebenläufigkeit $\rightsquigarrow$ Unbeeinflusster Zugriff durch mehrere, gleichzeitige Nutzer
- Störung $\rightsquigarrow$ Ausfall und Wiederherstellung von Ressourcen bleiben verborgen

Offenheit von verteilten Systemen

Verteilte Systeme als Vielzahl an *Diensten*

- Verhalten nach definierten Syntax- und Semantikregeln
- Kommunikation über Protokolle in Netzwerken
- Definition über Schnittstellen, z. B. mittels *Interface Definition Language* (IDL)

Spezifikationen von Schnittstellen

- Syntax $\rightsquigarrow$ Funktionsnamen, Parameter, Rückgabewerte, Fehler/Ausnahmen, ...
- Semantik $\rightsquigarrow$ Bedeutung (oft) in natürlicher Sprache beschrieben
- Gute Spezifikationen $\rightsquigarrow$ vollständig & neutral (ohne Implementierungsvorgaben)

Ziele von Offenheit

- Interoperabilität $\rightsquigarrow$ Systeme verschiedener Hersteller können zusammenarbeiten
- Portabilität $\rightsquigarrow$ Anwendungen/Komponenten laufen auf verschiedenen Systemen
- Erweiterbarkeit $\rightsquigarrow$ Einfaches Ersetzen oder Hinzufügen von Komponenten
- Flexibilität $\rightsquigarrow$ Einbindung alt. Betriebs- oder Subsysteme (z. B. Dateisysteme)

Beispiel: Web Services Description Language (WSDL)

- Technologie: XML-basiert, formale Schnittstellenbeschreibung
- Fokus: Operationen/Methoden
- Hauptnutzung: SOAP-basierte Webservices
- Eigenschaften:
  - Sehr formell, stark typisiert, detaillierte Serviceverträge
  + Standardisiert, formelle Verträge, viele Zusatzstandards verfügbar
  – Komplex, Overhead durch XML & SOAP-Header
- Einsatzgebiete: Unternehmensumgebungen, Finanzwesen, Gesundheitswesen, d. h. wo starke Verträge & Transaktionen wichtig sind

Beispiel: OpenAPI (Swagger)

- Technologie: JSON- oder YAML-basierte Spezifikation
- Fokus: Ressourcen/Endpunkte
- Hauptnutzung: RESTful APIs (HTTP, JSON)
- Eigenschaften:
  - Menschlich verständlich, locker typisiert, interaktive Dokumentation
  + Einfach, leichtgewichtig, geeignet für Webentwicklung
  – Weniger formelle Verträge, Sicherheit/Transaktionen nicht integriert
- Einsatzgebiete: Moderne Web-APIs, Microservices, mobile Apps, Cloud-Dienste

Demo: Taschenrechner als Web-Service

```yaml
openapi: 3.0.1
info:
  title: Calculator API
  description: API für einen Taschenrechner
  version: 1.0.0
servers: [{url: http://localhost:8000/}]
paths:
  /add:
    get:
      summary: Addiert zwei Zahlen
      operationId: app-calculator.add
      parameters:
        - in: query
          name: a
          schema: { type: integer }
          required: true
          description: Erste Zahl
        - in: query
          name: b
          schema: { type: integer }
          required: true
          description: Zweite Zahl
      responses:
        '200':
          description: Ergebnis der Addition
          content:
            application/json:
              schema:
                type: object
                properties:
                  result: { type: integer }
```

```python
import connexion

app = connexion.App(__name__, specification_dir=".")
app.add_api("calculator.yaml")


def add(a: int, b: int) -> dict:
    return {"result": a + b}


if __name__ == "__main__":
    app.app.debug = True
    app.run(host="localhost", port=8000)
```

Open in browser: http://localhost:8000/ui

Skalierbarkeit bzgl. Nutzern und Ressourcen

Limitierungen zentralisierter Lösungen

- Dienste $\rightsquigarrow$ Lokale Rechen- und Kommunikationskapazitäten sind begrenzt
  Beispiel: Ein Server für alle Nutzeranfragen wird zum Flaschenhals
- Daten $\rightsquigarrow$ Lokale Datenspeicherung und -verwaltung sind begrenzt
  Beispiel: Domain Name System (DNS) als Einzeltabelle funktioniert nicht
- Algorithmen $\rightsquigarrow$ Sammlung und Verarbeitung aller Informationen unmöglich
  Beispiel: Routing basierend auf vollständigen Informationen

Eigenschaften dezentraler Algorithmen

1. Kein Computer hat komplettes Wissen über den Systemzustand
2. Entscheidungen basieren auf lokalen Informationen
3. Ausfall eines Computers stört den Algorithmus nicht
4. Es wird kein global synchronisierter Takt (implizit) vorausgesetzt

Skalierbarkeit bzgl. Geographie und Verwaltungsdomänen

Herausforderungen beim Übergang von LANs zu WANs

- Local Area Networks (LAN):
  - Antwortzeiten im Bereich µs ⇝ synchrone Kommunikation
  - Zuverlässiges Broadcasting, z. B. für Dienstsuche
- Wide Area Networks (WAN):
  - Antwortzeiten im Bereich ms ⇝ synchrone Kommunikation blockiert
  - Interaktive Anwendungen erfordern besondere Sorgfalt und Geduld
  - Broadcast nicht zuverlässig ⇝ spezielle Ortungsdienste erforderlich
  - Kommunikation ist unzuverlässig und meist Point-to-Point

Herausforderungen bei mehreren unabhängigen Verwaltungsdomänen

- Konflikte bei Ressourcennutzung, Verwaltung und Sicherheit
- Vertrauen in eigene Domäne, aber nicht automatisch über Domänengrenzen
- Schutz vor Angriffen von außen ⇝ eingeschränkter Zugriff für Fremdnutzer
- Schutz der eigenen Domäne vor unsicherem (fremden) Code

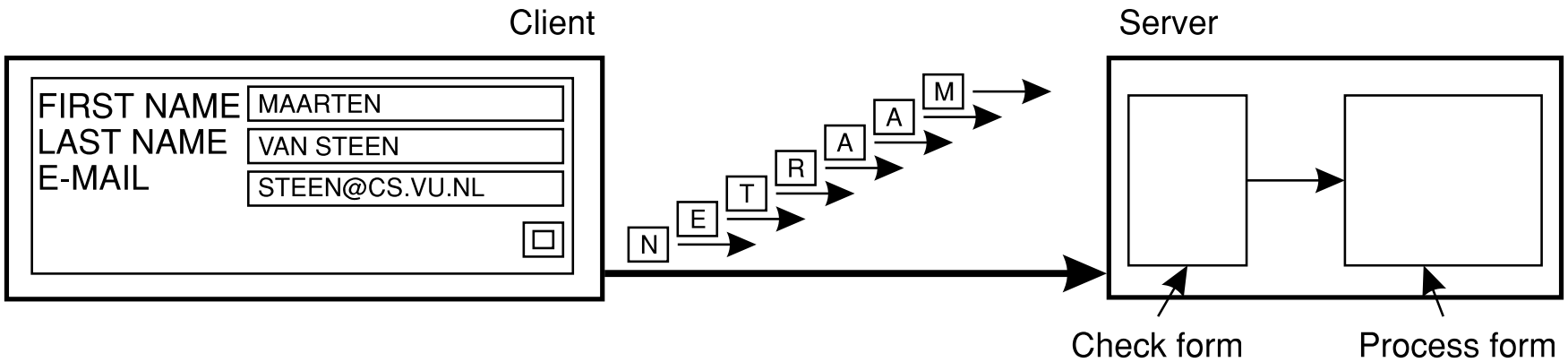
Lösungsansatz: Verstecken von Kommunikationslatenzen

Überbrücken von Antwortzeiten

- Erledigung anderer Aufgaben während des Wartes
- Nutzung von asynchroner Kommunikationstechniken:
  - Ausführung der Kommunikation in separaten *Threads*
  - Unterbrechung beim Antwortempfang mittels *Callbacks*

Vermeidung von Kommunikation

- Erledigung interaktiver Verarbeitung nahe beim Nutzer
- Beispiel: Formularcheck durch Server



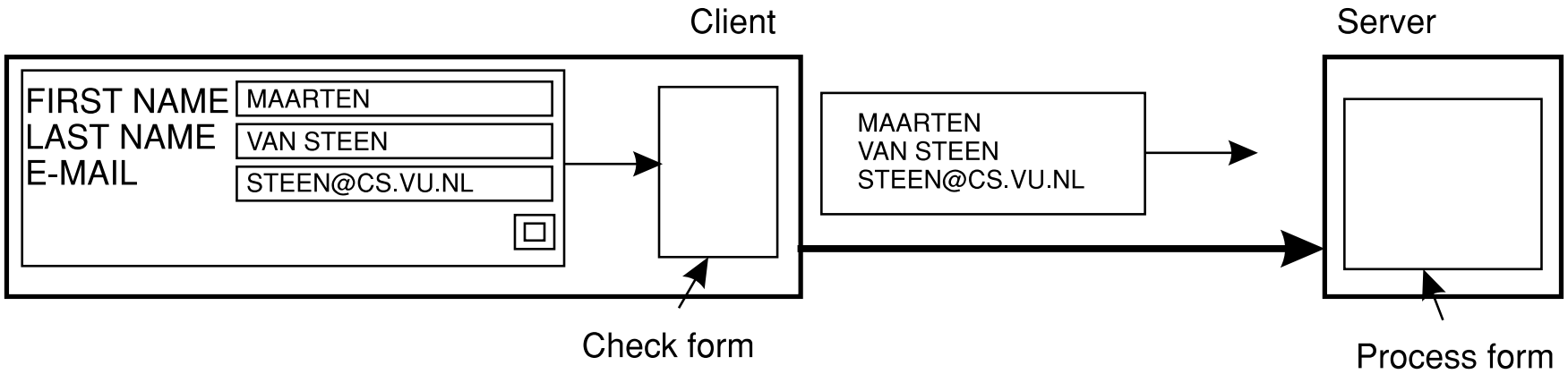
Lösungsansatz: Verstecken von Kommunikationslatenzen

Überbrücken von Antwortzeiten

- Erledigung anderer Aufgaben während des Wartes
- Nutzung von asynchroner Kommunikationstechniken:
  - Ausführung der Kommunikation in separaten *Threads*
  - Unterbrechung beim Antwortempfang mittels *Callbacks*

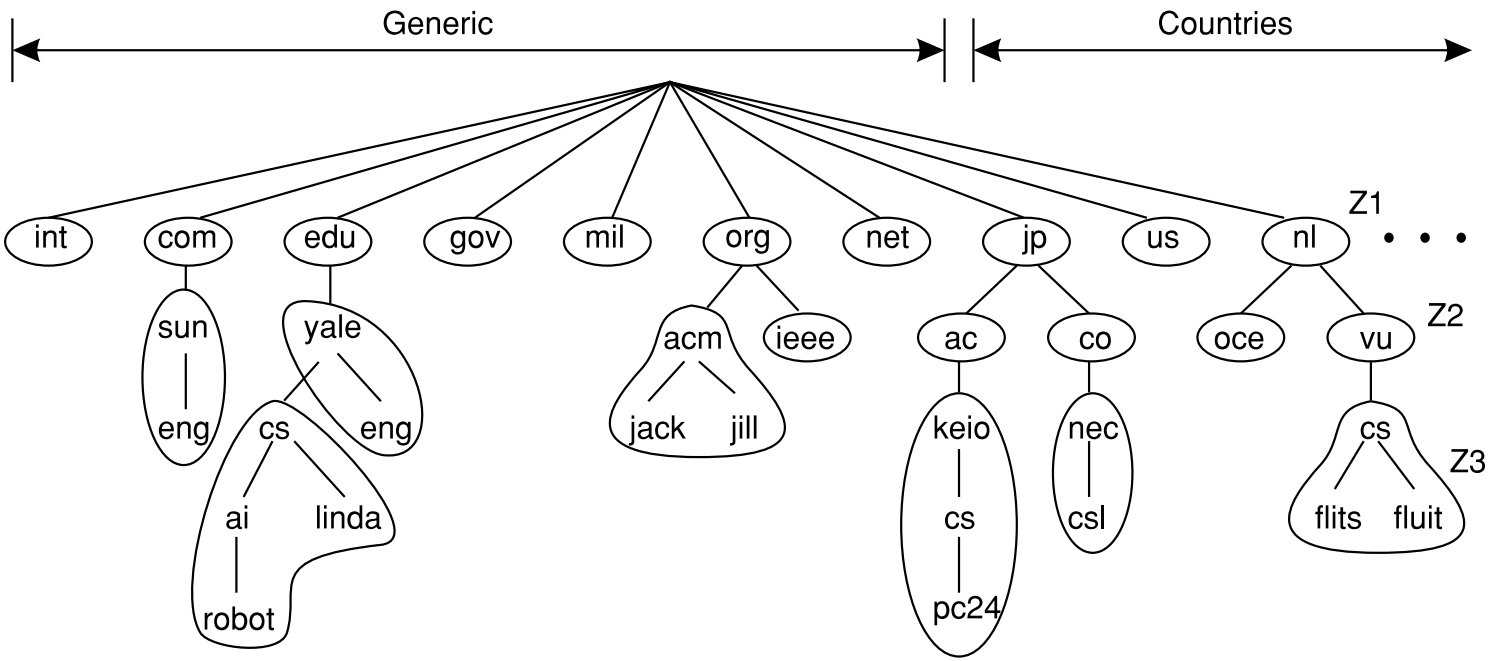
Vermeidung von Kommunikation

- Erledigung interaktiver Verarbeitung nahe beim Nutzer
- Beispiel: Formularcheck durch Client



Lösungsansatz: Aufteilung (klassisches *Teile und herrsche*)

- Zerteile eine Komponente und verteile sie über das gesamte System
- Beispiel: Domain Name System (DNS)



Lösungsansatz: Replikation und Caching

Replikation von Komponenten

- Erhöhung der Verfügbarkeit von Komponenten
- Lastverteilung zwischen Komponenten $\rightsquigarrow$ bessere Performance
- Kopie in Nähe des Nutzers $\rightsquigarrow$ Verringerung von Kommunikationslatenzen
- Entscheidung erfolgt durch Ressourcenbesitzer und meist geplant im Voraus

Caching von Ressourcen

- Speicherung der Kopie einer Ressource in der Nähe des Clients
- Entscheidung erfolgt durch Client und auf Abruf

Herausforderung: Konsistenz

- Kopien können auseinanderlaufen und Modifikation führt zu Inkonsistenzen
- Erlaubte Inkonsistenzen hängen vom Ressourcentyp ab, z. B.:
  - Webinhalte können (leichte) Verzögerungen tolerieren
  - Finanztransaktionen erfordern starke Konsistenz

Fehlerquellen und Komplexitätsursachen

Welche (falschen) Annahmen macht man ggf., wenn man zum ersten Mal ein verteiltes System entwickelt?

Fehlerquellen und Komplexitätsursachen

Welche (falschen) Annahmen macht man ggf., wenn man zum ersten Mal ein verteiltes System entwickelt?

Fälschliche Annahmen (nach Peter Deutsch)

- Das Netzwerk ist zuverlässig.
- Das Netzwerk sicher.
- Das Netzwerk homogen.
- Die Netzwerktopologie verändert sich nicht.
- Die Latenz ist 0.
- Die Bandbreite ist unendlich.
- Die Transportkosten sind 0.
- Es gibt nur einen Administrator.

Inhalt

1. Einführung
    Ziele von verteilten System
    Arten von verteilten Systemen

Arten von verteilten Systemen

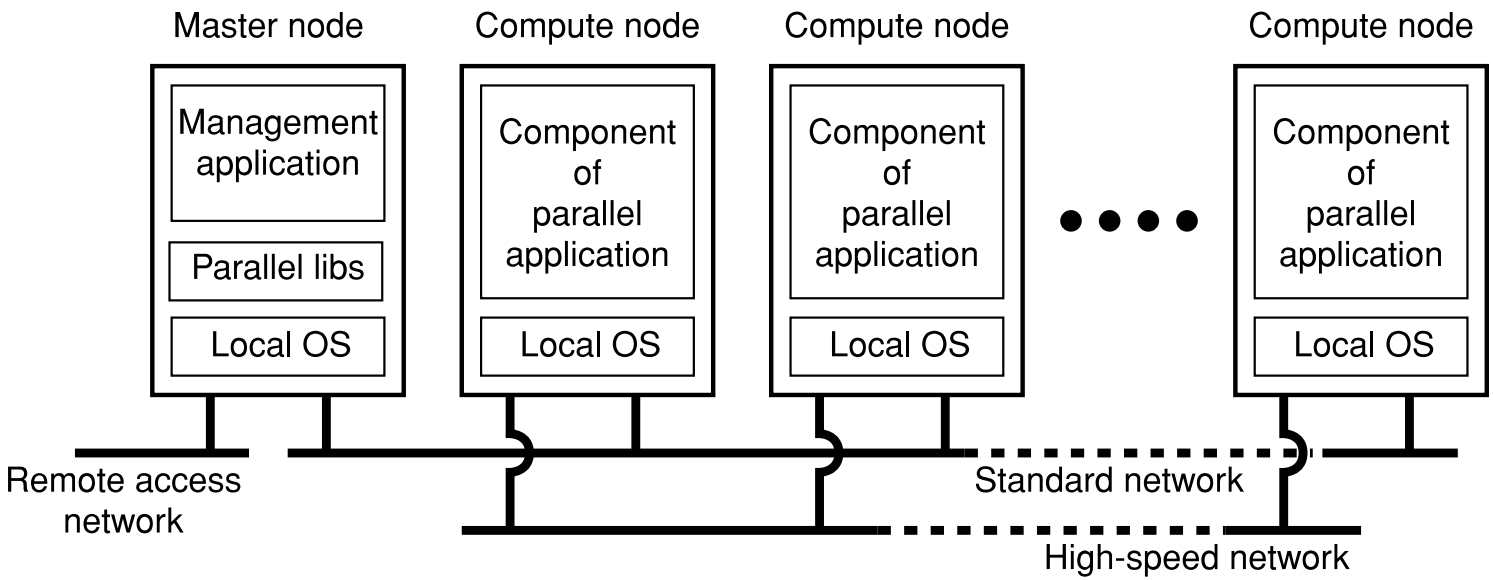
1. Verteilte Rechensysteme
   - Cluster Computing Systeme
   - Grid Computing Systeme
2. Verteilte Informationssysteme
   - Verteile Transaktionssysteme
   - Enterprise Anwendungsintegration
3. Allgegenwärtiges verteiltes Computing
   - Heimautomatisierung
   - Elektronische Gesundheitssysteme
   - Sensornetzwerke

Verteilte Rechensysteme: Cluster Computing Systeme

- Preis-Leistungs-Verhältnis von Personalcomputern (PC) verbesserte sich

⇝ Supercomputer aus Standardhardware und Hochgeschwindigkeitsnetzwerken

- Homogenität ⇝ gleiche Hardware, Netzwerke, Betriebssystem

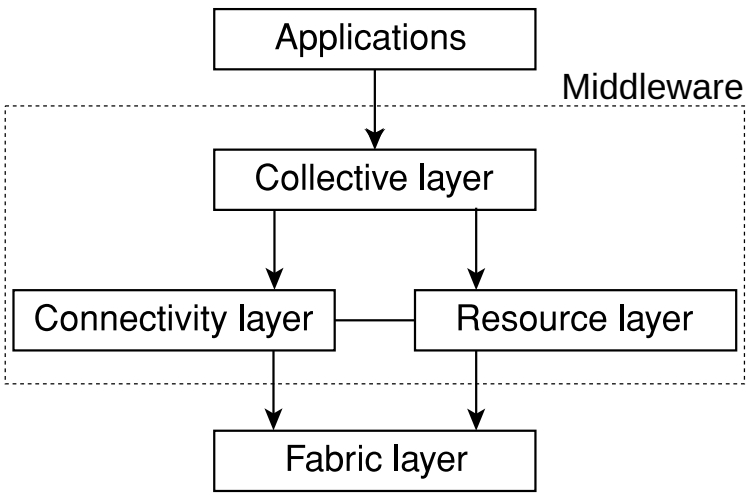


Verteilte Rechensysteme: Grid Computing Systeme

- Heterogenität ⇝ unterschiedliche Hardware, Netzwerke, Betriebssysteme
- Mehrere Verwaltungsdomänen zusammengefasst zu virtuellen Organisationen
- Ressourcen: Rechenleistung, Speicher, Datenbanken, spezialisierte Geräte, ...

Schichtenarchitektur

1. Fabric ⇝ Schnittstellen zu lokalen Ressourcen
2. Connectivity ⇝ Kommunikations- und Sicherheitsprotokolle, Authentifizierung und Rechte-Delegation
3. Resource ⇝ Verwaltung einzelner Ressourcen, Zugriffskontrolle
4. Collective ⇝ Verwaltung mehrerer Ressourcen, z. B. Ressourcenentdeckung, Aufgabenplanung, Datenreplikation
5. Application ⇝ Anwendungen innerhalb virtueller Organisationen



Verteilte Informationssysteme

- Unternehmensweite Integration verschiedener Netzwerkanwendungen

Verteilte Transaktionssysteme

- Unternehmensanwendungen basieren zumeist auf Datenbanken
- Zugriffe erfolgen mittels Transaktion nach dem Alles-oder-Nichts-Prinzip

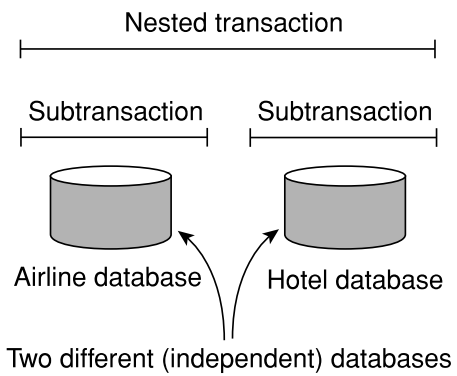
⇝ ACID-Eigenschaften: Atomar, Konsistent, Isoliert, Dauerhaft

- Verteilte Anwendungen fassen Zugriffe zu verteilten Transaktionen zusammen

Ausführung verteilter Transaktionen

- Aufteilung einer Haupttransaktion in mehrere parallele Subtransaktionen
- Änderungen von Subtransaktionen werden erst bei Commit der Haupttransaktion dauerhaft
- Bei Abbruch der Haupttransaktion werden alle Subtransaktionen zurückgenommen
- Ermöglicht verteilte Bearbeitung großer Transaktionen auf mehreren Computern

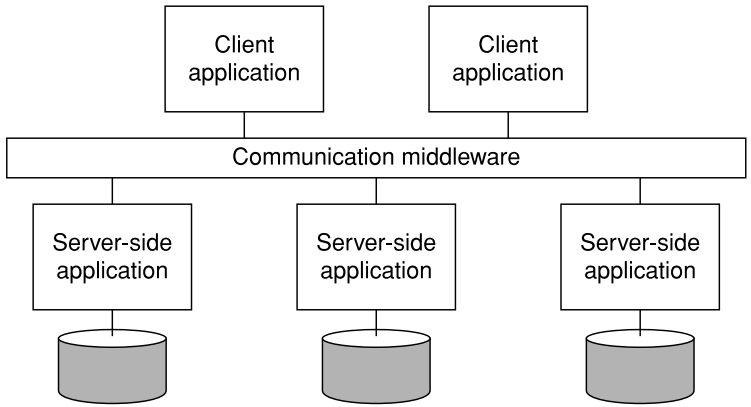
Beispiel: Datenbanken



Enterprise Anwendungsintegration

- Zunehmende Entkopplung von Anwendungen und Datenbanken

⇝ Direkte Kommunikation zwischen Anwendungskomponenten erforderlich



Remote Procedure Call (RPC) bzw. Remote Method Invocation (RMI)

- Lokale Prozedur-/Methodenaufruf werden als Nachricht versendet

– Gleichzeitige Verfügbarkeit von Aufrufer und Aufgerufenem

– Enge Kopplung der Komponenten

Message-Oriented Middleware (MOM)

- Senden von Nachrichten an logische Kontaktpunkte (Topics/Subjects)
- Abonnieren von Nachrichtenarten (Publish/Subscribe)

\+ Entkoppelte Sender und Empfänger

\+ Asynchrone Kommunikation

Allgegenwärtiges verteiltes Computing

Stabilität traditioneller verteilte Systeme

- Feste Knoten mit dauerhafter, hochwertiger Netzverbindung
- Einsatz von Techniken zur verteilten Transparenz und Fehlerbehandlung

Merkmale allgegenwärtiger verteilter Systeme

- Instabilität als Normalfall $\rightsquigarrow$ mobile, batteriebetriebene, drahtlose Geräte
- Wenig bis keine manuelle Kontrolle

$\rightsquigarrow$ Automatische Umgebungsentdeckung und Anpassung erforderlich

- Generelle Anforderungen
  - Umgebungen können sich jederzeit ändern, z. B. Verfügbarkeit von Netzwerken
  - Nutzungsarten können wechseln $\rightsquigarrow$ einfache bzw. automatische Konfiguration
  - Fokus auf Informationsabfrage und -bereitstellung
- Verzicht auf verteilte Transparent $\rightsquigarrow$ Verteilung von Daten, Prozessen und Kontrolle wird nicht mehr versteckt sondern explizit behandelt

Beispiel: Sensornetzwerke

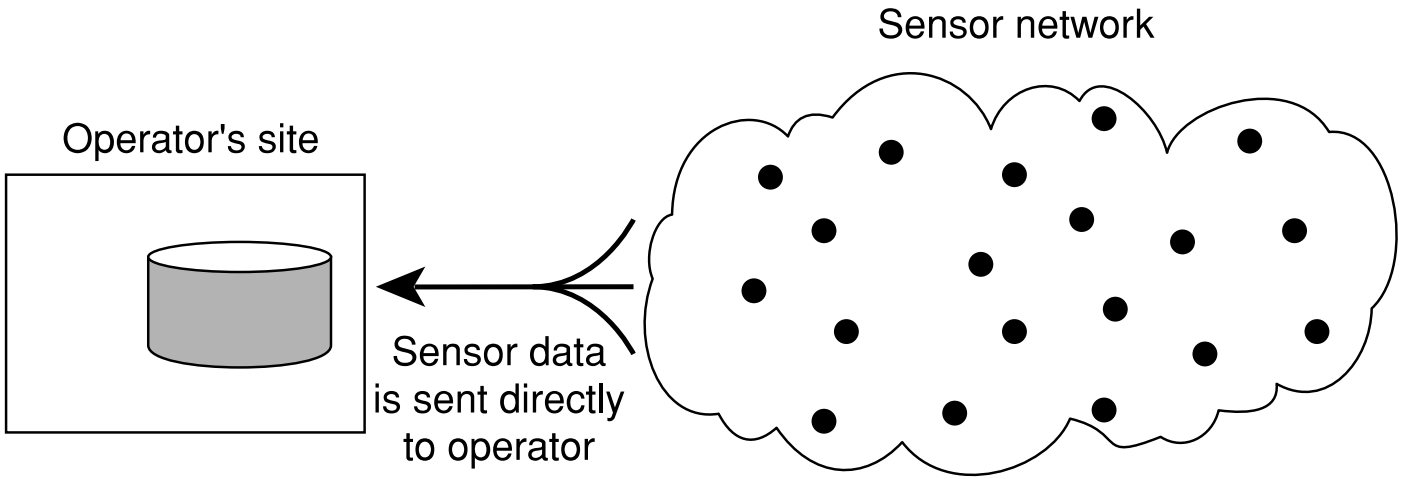
Aufbau und Eigenschaften

- Hunderte bis Tausende kleine Sensorknoten mit Messgeräten
- Meist batteriebetriebene Geräte mit kabelloser Kommunikation

⇝ Ressourcenbeschränkt bzgl. Energie, Rechenleistung, Kommunikation

Extremfälle der Organisation

- Zentralisierte Datensammlung ⇝ hoher Ressourcenverbrauch



Beispiel: Sensornetzwerke

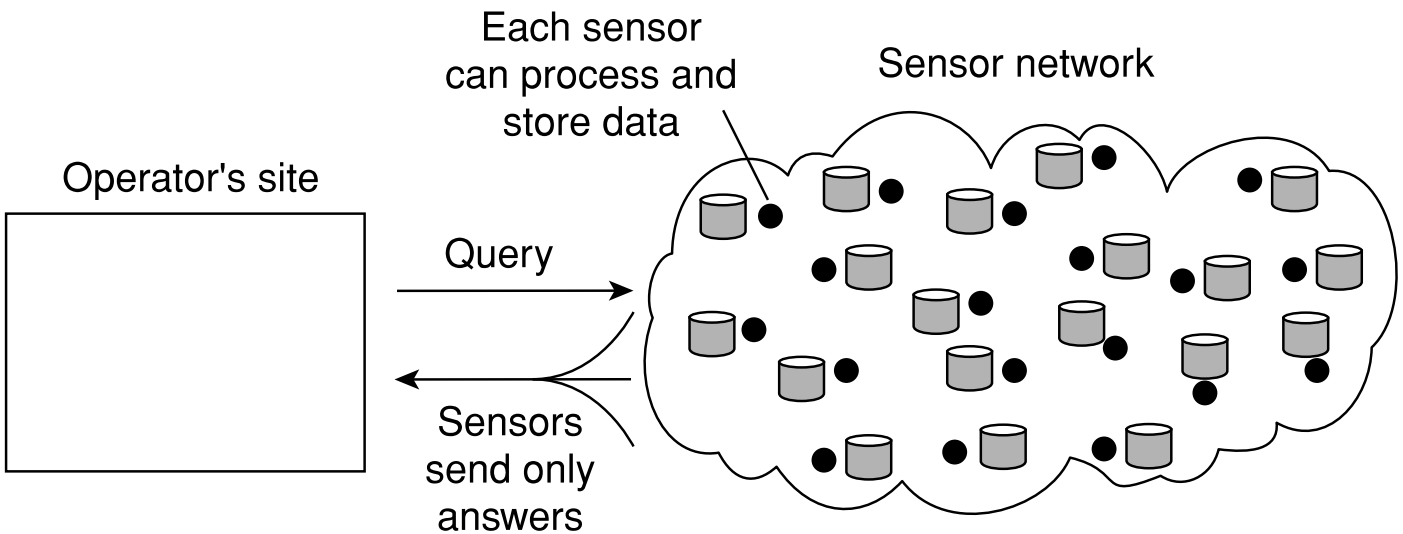
Aufbau und Eigenschaften

- Hunderte bis Tausende kleine Sensorknoten mit Messgeräten
- Meist batteriebetriebene Geräte mit kabelloser Kommunikation

⇝ Ressourcenbeschränkt bzgl. Energie, Rechenleistung, Kommunikation

Extremfälle der Organisation

- Vollständige Verteilung ⇝ möglicher Datenverlust



Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz